PRATHYUSHA ENGINEERING COLLEGE

(An Autonomous Institution)

TIRUVALLUR - 602025

“In-Memory Computing for Artificial Intelligence”

CASE STUDY

OF

241GES301

DIGITAL PRINCIPLES AND COMPUTER ORGANISATION

The Bachelor of Engineering (B.E.)

B.E. - CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

Vedha Prakash (111425148079)

Vemula Barath (111425148080)

Vinesh (111425148081)

Vinotha (111425148082)

Deekshita (111425148083)

Yuvaraj (111425148084)

Faculty In-Charge

A. PRIYA DHARSHINI

Assistant Professor

1. Abstract

In-memory computing (IMC) is a computing paradigm in which certain computations are performed directly inside the memory array where data is stored, instead of repeatedly moving data between separate memory and processing units. Because modern artificial intelligence (AI) workloads, especially deep neural networks, are dominated by matrix-vector multiplications over large stored weight matrices, they are a natural fit for this approach. Memory arrays built from SRAM, DRAM, resistive RAM, phase-change memory and similar devices can perform multiply-accumulate operations in place, reducing data movement, energy consumption and latency. This case study explains the motivation for IMC, its organization, memory technologies, execution model, applications in AI, advantages and limitations.

2. Introduction

Conventional computers follow the von Neumann architecture, in which a processor fetches data from a physically separate memory, computes on it and writes results back. Over several decades, processor speed has improved much faster than memory access speed and energy efficiency. This gap is known as the *memory wall* or von Neumann bottleneck. In many systems, moving a value from off-chip DRAM costs far more energy than performing an arithmetic operation on it.

Neural networks make the problem worse: models contain millions to billions of weights, and each inference or training step touches most of them. In-memory computing addresses this by keeping the weights stationary inside the memory and bringing the computation to them. This report studies how that idea works and why it matters for AI hardware.

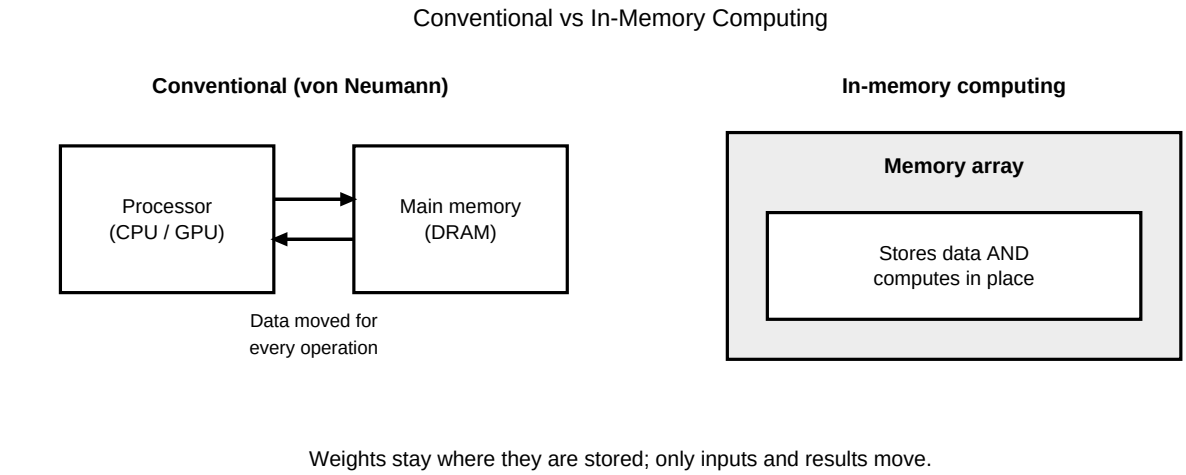


Figure 1. Original simplified comparison of conventional and in-memory computing.

3. Objectives

- To understand the motivation for in-memory computing.
- To study the organization of a memory crossbar array.
- To understand the memory technologies used for IMC.
- To compare von Neumann and in-memory architectures.
- To examine AI applications, advantages and limitations.

In-Memory Computing Architecture

The core structure of analog IMC is the *crossbar array*: a grid of horizontal word lines and vertical bit lines with a memory device at every crossing. Each device stores a value as its conductance *G*. A neural-network weight matrix is mapped onto these conductances. Input values are applied to the word lines as voltages *V*. By Ohm's law each device produces a current equal to *V x G*, and by Kirchhoff's current law the currents on each bit line add up. The current on bit line *j* is therefore the dot product of the input vector with column *j* of the weight matrix, so the whole matrix-vector multiplication happens in a single step.

Resistive Crossbar Performing a Matrix-Vector Multiplication

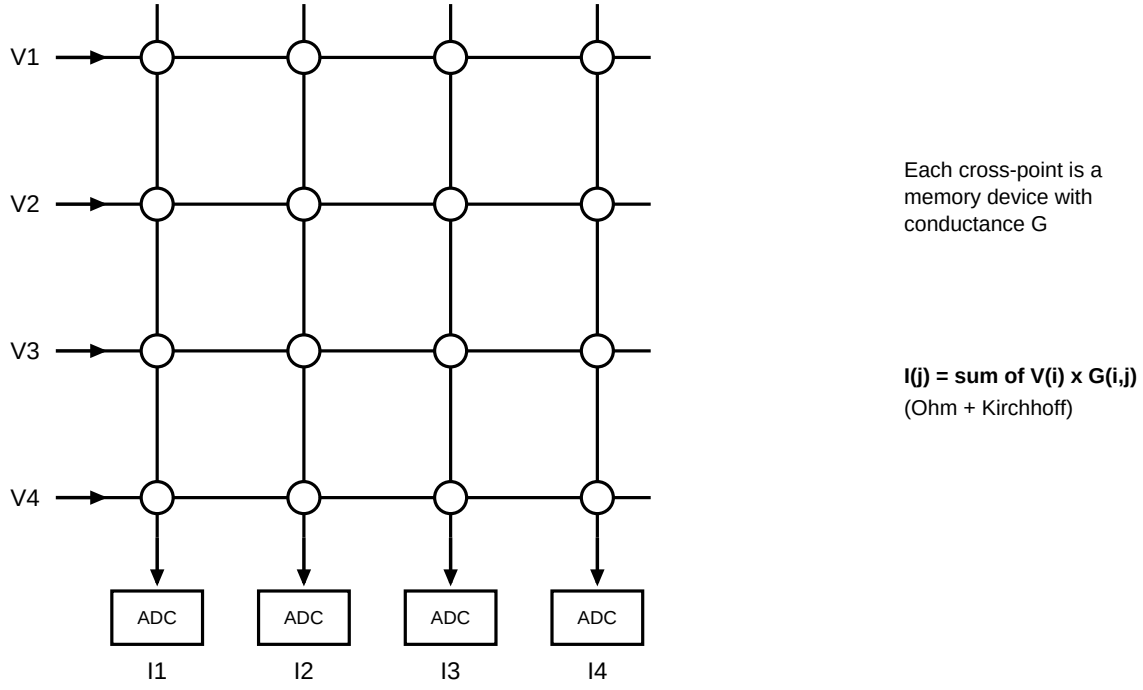


Figure 2. Simplified resistive crossbar performing a matrix-vector multiplication.

3.1 Peripheral Circuits

The array is surrounded by circuits that make it usable: digital-to-analog converters (DACs) or pulse drivers to apply inputs, analog-to-digital converters (ADCs) to read the summed currents, and digital logic for activation functions, normalization and control. These peripherals often account for a large part of the area and energy of an IMC chip.

3.2 Memory Technologies for IMC

Several memory technologies can be used to build IMC arrays. They differ in density, speed, endurance and how naturally they support analog computation.

Technology	How it computes	Notable characteristics
SRAM	Bit-line charge or current accumulation; digital or mixed-signal compute in the cell array	Fast, mature CMOS process, high endurance; low density and volatile
DRAM	Bulk bitwise operations by activating multiple rows; logic near banks	Very high density; limited precision of in-array operations; needs refresh
ReRAM / Memristor	Analog conductance stores weights in a crossbar	High density, non-volatile; device variability and limited endurance
Phase-change memory (PCM)	Analog conductance of amorphous/crystalline states	Multi-level storage; conductance drift over time
MRAM	Resistance states of magnetic tunnel junctions	Good endurance and speed; small resistance window
Flash / FeFET	Threshold voltage or charge stores weights	Non-volatile and dense; slower and costlier to program

Table 1. Common memory technologies explored for in-memory computing.

3.3 Processing-in-Memory vs In-Memory Computing

The terms are often used loosely. *Near-memory* or *processing-in-memory (PIM)* designs place digital processing logic next to or inside a memory chip or stack, as in commercial HBM-PIM and GDDR6-based PIM products. *In-memory computing* in the strict sense uses the memory cells themselves as compute elements, as in crossbars. Both reduce data movement; they differ in how deeply compute is merged with storage.

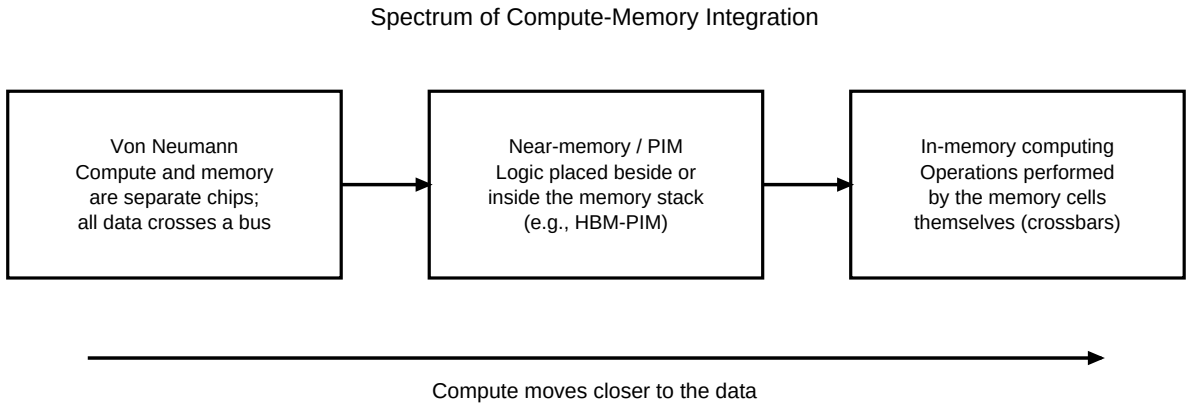


Figure 3. Spectrum from von Neumann to in-memory computing.

4. Von Neumann vs In-Memory Computing

Von Neumann systems are flexible and precise: they run any program with exact digital arithmetic, but pay a cost for moving data. In-memory systems are specialized: they excel at dense, repetitive multiply-accumulate work and sacrifice some generality and precision to avoid data movement.

Aspect	Von Neumann (CPU/GPU)	In-memory computing
Data movement	Weights and activations move between memory and processor	Weights stay stationary in the array
Matrix-vector multiply	Many sequential or parallel multiply-add steps	One parallel step per array read
Precision	Exact digital (FP32, FP16, INT8)	Limited (about 4-8 bits effective in analog designs); noise-sensitive
Flexibility	General-purpose programmable	Best for fixed matrix-heavy workloads
Energy per operation	Dominated by memory access	Potentially much lower for suitable workloads
Maturity	Very mature tools and ecosystem	Emerging; limited software support

Table 2. Comparison of design goals.

5. In-Memory Execution Model

In a typical heterogeneous system, a host CPU runs the application and offloads neural-network layers to an IMC accelerator. A trained model is first mapped: weight matrices are split into tiles that fit the array size and programmed into the memory devices. During inference, input activations are fed to the arrays, the arrays produce analog partial sums, ADCs digitize them, and digital units combine partial sums and apply activation functions before passing data to the next layer.

Because weights are written once and read many times, inference is the strongest use case. Programming (writing) devices is slower and may wear them out, so frequently changing weights, as in training, are harder to support.

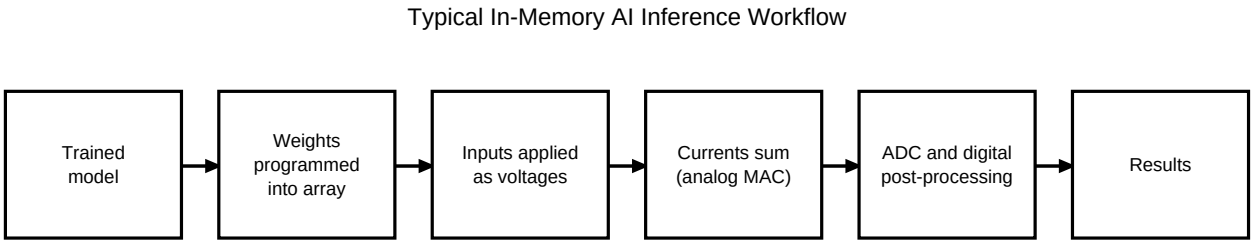


Figure 4. Typical in-memory AI inference workflow.

6. Role of In-Memory Computing in AI

6.1 Deep Neural Network Inference

Fully connected and convolutional layers reduce to matrix-vector or matrix-matrix multiplication. Crossbars execute these directly, making them attractive for image recognition, speech and other inference workloads. Research chips, such as IBM's multi-core phase-change-memory design, have demonstrated neural-network inference on analog IMC hardware.

6.2 Transformers and Large Language Models

Transformer models contain large projection and feed-forward matrices and spend much of their time waiting on memory bandwidth. PIM approaches that place compute near DRAM or HBM are being studied and productized to speed up memory-bound stages such as attention and token generation.

6.3 Edge and Low-Power AI

Battery-powered devices such as sensors, wearables and cameras need AI at very low energy. IMC can keep the model on-chip in non-volatile memory and avoid costly off-chip accesses, which suits always-on keyword spotting, vision and anomaly detection.

6.4 Neuromorphic and Brain-Inspired Computing

Memory devices with tunable conductance resemble biological synapses. Arrays of such devices are used to build spiking and other brain-inspired networks, as well as associative memories.

6.5 Data-Intensive Workloads

Recommendation systems, graph analytics, database scans and genome analysis are limited by memory access. Commercial PIM products, including UPMEM DIMMs and HBM-PIM from Samsung, target such memory-bound workloads.

7. Advantages

- Greatly reduced data movement between memory and processor.
- High parallelism: an entire matrix-vector product in one array operation.
- Potentially large gains in energy efficiency and throughput for inference.
- Non-volatile devices allow on-chip model storage and low standby power.
- Well matched to edge AI where power and latency are tight.

8. Limitations

- Analog noise, device variability and conductance drift limit accuracy and precision.
- ADCs and DACs add area, energy and latency overhead that can offset the gains.
- Limited write endurance and slow programming make training and frequent weight updates difficult.
- Fixed array sizes require tiling large models and managing partial sums.
- Immature compilers, programming models and benchmarks compared with GPUs.
- Manufacturing maturity and cost of emerging memory devices.

9. Key Findings

Aspect	Finding
Motivation	Data movement, not arithmetic, dominates energy and time in AI workloads (the memory wall).
Architecture	Memory crossbars compute matrix-vector products using Ohm's law and Kirchhoff's current law.
Technology	SRAM, DRAM, ReRAM, PCM, MRAM and Flash offer different density, endurance and precision tradeoffs.
Execution	Weights are programmed once; inputs are applied repeatedly, with ADCs and digital logic completing each layer.
Applications	DNN inference, transformers, edge AI, neuromorphic systems and memory-bound data workloads.
Performance	Gains depend on device quality, ADC overhead, array utilization and how well the model maps to the hardware.

10. Conclusion

In-memory computing challenges the traditional separation of processing and storage by performing computation where data resides. For AI, whose core operation is multiplying large stored matrices by input vectors, this can sharply reduce the data movement that limits conventional processors. Analog crossbars, digital SRAM-based macros and near-memory PIM products each explore a different point in this design space. Challenges remain in accuracy, device reliability, peripheral overhead, training support and software tools. In practice, IMC accelerators will complement CPUs and GPUs rather than replace them: CPUs provide general-purpose control, GPUs provide flexible high-throughput compute, and IMC units accelerate the memory-bound, matrix-heavy parts of AI workloads with high energy efficiency.

11. References

1. A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, E. Eleftheriou, “Memory devices and applications for in-memory computing,” *Nature Nanotechnology*, vol. 15, 2020.

2. D. Ielmini and H.-S. P. Wong, “In-memory computing with resistive switching devices,” *Nature Electronics*, vol. 1, 2018.

3. A. Shafiee et al., “ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars,” *ISCA*, 2016.

4. P. Chi et al., “PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-based Main Memory,” *ISCA*, 2016.

5. M. Le Gallo et al., “A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference,” *Nature Electronics*, vol. 6, 2023.

6. M. Horowitz, “Computing's energy problem (and what we can do about it),” *IEEE ISSCC*, 2014.

7. S. Lee et al., “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology,” *ISCA*, 2021.

8. O. Mutlu et al., “A Modern Primer on Processing in Memory,” 2020.

9. Figure note: All figures in this report are original simplified diagrams created for educational use; they are not specific chip floorplans.